

ECS5311 - Advanced Digital Logic Notes

Chris Lin
chris_lin1@baylor.edu

June 28, 2026

Contents

1	Diodes & Semiconductor Physics (8/27)	2
2	Doping & the P-N Junction (8/29)	4
3	BJTs & Diode/Transistor Logic (9/3)	7
4	FETs, MOS & Static CMOS (9/5)	8
5	Static CMOS Timing (9/10)	11
6	Design Techniques (9/12)	12
7	Online Lecture (9/17)	14
8	SR Latch & Race Conditions (9/19)	14
9	Latch-free Async Design (9/24)	15
10	State Reduction & Assignment (9/26)	17
11	Glitch-free Design & Timing (10/1)	18
12	Clock Domains & Verilog Intro (10/3)	20
13	Blocking/Nonblocking & the Barrel Shifter (10/8)	21
14	Nexys Setup & Barrel Shifter Build (10/10)	23
15	Parameterized Barrel Shifter (10/15)	25
16	Board Interface & Simulation (10/17)	26
17	Ripple Carry Adder (10/22)	27
18	Adder Test Bench (10/24)	28
19	Carry Lookahead Adder (10/29)	30
20	Block Carry Lookahead (10/31)	31
21	Project Design Methodology (11/5)	33

22 Bit Reversal & LFSR (11/12)	34
23 Conditional Sum Adder (11/14)	34
24 File-driven Test Benches (11/21)	35
25 Conditional Sum Wiring & Recursion (12/3)	36
26 Babbage Difference Engine (12/5)	37
27 Difference Engine Implementation (12/10)	39

1 Lecture 1: Diodes & Semiconductor Physics (Tuesday 8/27/2024)

Definition 1 (Diode). *Allows 1-direction current*

- .7V drop common due to silicon; the drop changes due to material.
- For semiconductors to work, we need 4 bonds on the molecule.
- Made of 2 types of materials; P and N type materials.
 - Technically made of a P-N junction.
 - 3 diodes in a transistor (NPN or PNP)
 - Determine whether to allow it to be controlled by current or field.
- Look at Schroedinger's wave equation
 - Know the wave is proportional to energy
 - Can get likelihood if we square the ψ
 - Begin with hydrogen, only 1 electron so 1 interaction
- How do we determine electric force between 2 charge carriers?
 - Force equation between two masses, like gravity
 - See figure 1
- How do we go from force to energy? Integrate over the force equation: we end up with $\frac{1}{r}$
 - See fig 2
 - How should we plot this? We need to figure out r_1, r_2 for the integral, picking different break points
 - 13.6 eV: lowest state in the hydrogen atom, a good point of reference
 - * Call this the **ionizing break point**
 - -3.4 eV: next breaking point
 - 2 electrons can sit at the base state; every energy state can hold up to 2
 - * Fermions have spin one-half; this allows an atom to hold two: one spin up, the other spin down
 - * Bosons can have infinitely many at an energy level
 - If I add a third electron, this creates a *band*
 - * The jump from 13.6 to 3.4 creates a band gap

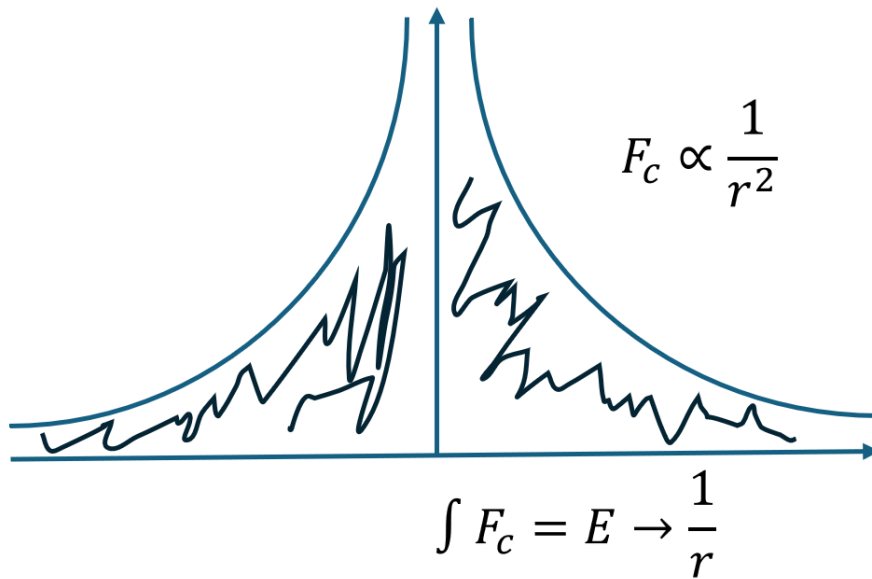


Figure 1: Force Plot

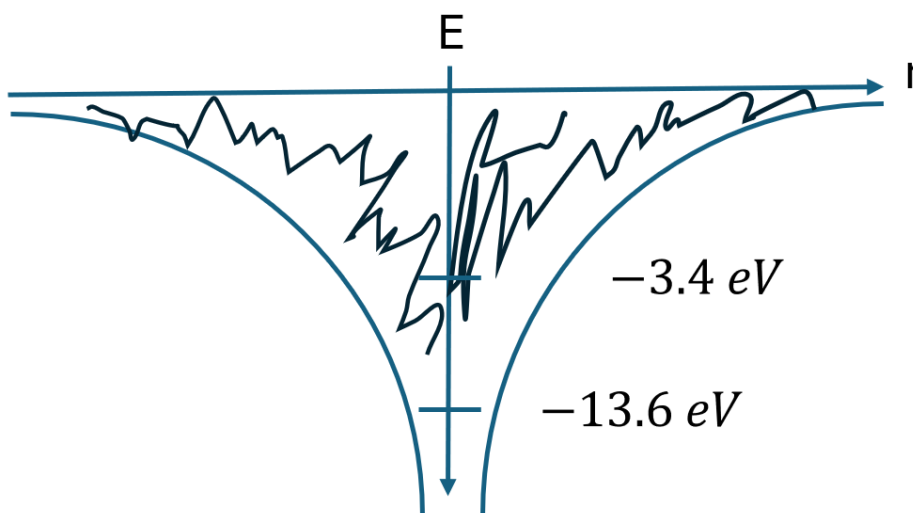


Figure 2: Energy Plot

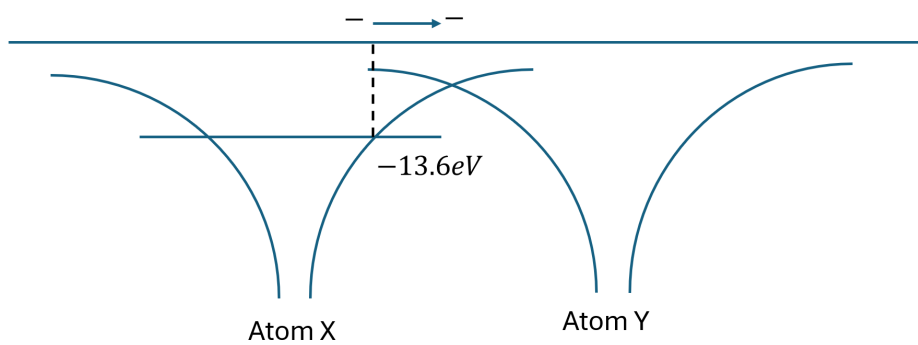


Figure 3: electron jump

- * See fig 3
- * Atom A also shares some electron with atom B

- * At position 2, there is much less of an energy drop
- * If you move the two atoms together, the energy required to move is even lower
- * This will result in bifurcation and split, thus spreading energy and keeping all 4 positions
- * By the Heisenberg uncertainty principle, we don't know exact position and velocity; we know $\Delta p \cdot \Delta x \geq \hbar$, which has nothing to do with how you observe, just that you observe
- * As atoms are brought together, the wave functions from both add together
- * Closer to the center, we get a lower energy state; bands result from Pauli exclusion and force when bringing atoms together
- This is how we get a **valence band**; take the lowest energy band available

2 Lecture 2: Fermi Energy, Doping & the P-N Junction (Thursday 8/29/2024)

•

Definition 2 (Fermi energy). Gives the "probability to be in a conduction or valence band"

- Intrinsic semiconductors
 - Column 4 of the periodic table
 - Gap between valence and conduction is small
 - 4

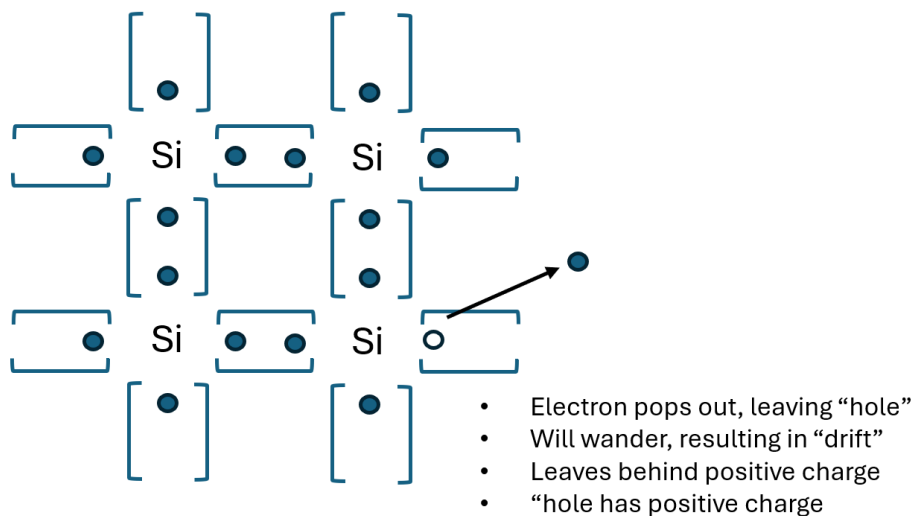


Figure 4: electron drift

- Bonding drops to a lower energy state
- One electron in the bond will wander off, leaving a hole
- The hole and electron are both considered carriers
- If another electron pops off and fills the hole, it will look like the hole moved
 - * Net effect is the hole changing locations; hence holes are also carriers
 - * This is why holes move slower than electrons

- P-type flow is much slower than N-type due to a difference in which carriers are *moving*
- We can boost the number of carriers
 - Introduce new elements
 - 5

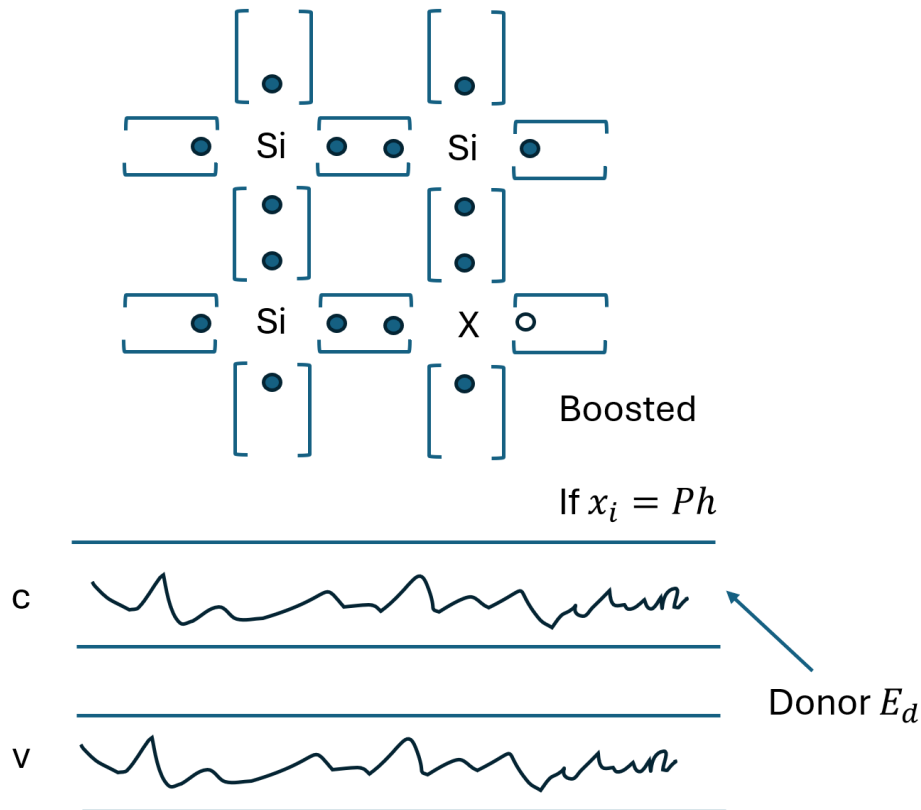


Figure 5: electron boost; ex Ph

- Fermi energy moves up because the energy needed for an electron to move is lowered significantly
 - * Now, the energy to create holes is also lower
- Electrically neutral, even though the electron is the charge carrier
 - * Get majority carrier e^-
- We call this **extrinsic**; we have doped with phosphorus (or any element that gives extra electrons)
- So why not add an element that introduces even more electrons?
 - Adding more and more electrons means adding elements that are nonmetals
 - Adding these elements results in more protons in addition to more electrons; more protons pull the shell down to a lower energy; now the energy needed to jump between donor states is even more
- An electron moving up/down the gap emits photons; this is how **LEDs** work
- If we use boron (3 electrons on the outer shell), it is only held together by 3, so fewer bonds are held together; now the hole is closer to the top of the valence band and the Fermi energy is also much lower. We will now have a higher chance of the hole moving; boron's majority

mover is a hole. There will need to be holes moving from boron to boron, which looks like a hole flowed to a silicon

- Boron results in + holes as the majority carrier; phosphorus results in e^- electrons as the majority carrier
- img3
- When P is placed next to N, electrons can move into P, resulting in a true net positive and net negative in a region; this is called the depletion region (depleted of charge carriers)
 - The depletion region fills really fast
 - This is called an insulator; this is a diode
 - There is a voltage differential due to this local movement
- img5
- What happens if I connect P to N via a battery?
 - It will try to push electrons in one direction (holes in another)
 - It will force electrons from P and N to jump; call this **forward bias**
 - Reduces the depletion zone; with enough voltage from the battery, the depletion zone disappears and it becomes a conductor
 - * .7V or more
- img6
- Reverse bias
 - Battery flipped; it tries to force electrons. There will be some flow, but it results in the depletion region increasing
 - Call this the "avalanche region" (view graph, negative flow)
- img7
 - Can B be negative V? No
 - Can B be positive V? Yes; if we hook up a voltage source at the end
 - The diode prevents output from being .3V (the minimum)
 - If we know .3V at B, what do we know?
- img8
 - If input is 0, it just wants something to be $> -.7V$
 - Need a pull-down resistor; allows Z to become 0
 - Use Ohm's law to determine the resistor needed: $V = IR$
 - For 0,1, we will pick $\max(.7, 5.7)$ for the agreement region; .7 (in ii) results in 0
 - In ii, introduce a pull-up resistor
 - This is diode logic; the downside is the voltage differential results in decreasing voltage

3 Lecture 3: BJTs, Diode Logic & Gate Construction (Tuesday 9/3/2024)

- Review of last time: P/N results in a diode
- What if we add another N in front of P? We get a new depletion zone
- img1
- With enough charge carriers through via forward bias, it will remove the depletion zone, leak over, and remove the other depletion zone; this results in essentially a wire
 - If I lower the forward bias, there are fewer charges and flow eventually stops; similar to *variable resistance*, but not a resistance "on one area here but over there", this is considered a trans-resistor or *transistor*
 - Can also be used for amplification, but we are only focused on using this as a *switch*
 - There can also be electro-mechanical switches (relay), but there are many advantages of an electrical switch
 - * Speed, lower power, more reliable, smaller, etc
- img2
- Photo-resist layer → introducing light results in P/N wells with each iteration
 - All set in N type material; prevent things from flowing. Isolates N, prevents leakage
- We will redraw since we only use the left side; this is a BJT
- Another way to construct a BJT: img3

c	c	c
0	0	0
0	1	0
1	0	0
1	1	V-.7

- Logic: any diode with 1 is essentially not there. Thus, 1|1 results in resistor pull-up
- Note, output is not very strong. Similar to diode logic, controlled by the original signal. Not as strong as the original signal, but not limited to only one configuration.
- We have an AND gate, but now we want to amplify
 - How do we build an amplifier?
- img3
 - If A is high (say 5V), then B flows/turns on ($5 - .7 = 4.3V$ output). This will burn out, so we need to stop the current
 - * Easiest with a resistor
 - * A resistor on ground will raise the voltage, which we don't want because it changes the output
 - * Thus, we put a resistor at the beginning right after A inflow
 - * Low resistance pulls down to 0 very hard; gives us a good zero
 - * Resistor A for current limiting

- * If nothing is flowing through, the ground/bottom connection is ignored, so output to B is whatever is hooked up to B
- * If we attach a pull-up resistor, it won't affect pulldown
- * Strong outflow; this is an inverter amplifier

- img4

- Now take our previous AND gate and combine it with the inverter amplifier
- We know $D = A \text{ AND } B, C = \text{NOT}(D)$
- Now we have a NAND gate, a universal gate
 - * Anything can be written as a sum of products; this is why NAND is considered universal, same as a NAND leading into another NAND
 - * The same can be done with a NOR gate
- NAND is built with NPN, NOR with PNP. NPN is better, which is why NAND is used more commonly than NOR. It is slightly faster because we primarily use electrons vs holes

- With more and more gates, we get more current flowing in; let's say a 16-input AND gate. One transistor allows lower latency, but also means more current coming in. Too much and it can overload.

- img5

- This is a totem pole
- We want to either turn on the bottom half and hook to ground or turn on the top and hook up to power

A	B		T_1	T_2		C
0	0		Off	Off		Z
0	1		Off	On		0
1	0		On	Off		$A - 1.4$
1	1		On	On		0 (I, high current)

- What if I had a middle zone? Introduce a resistor right after A and B, R_1 and R_2
 - For $D = 1$, we get resistors, AKA a resistor divider $\frac{\max(R_1, R_2)}{R_1 + R_2}$
 - Now for $D = 0$, we get $A = 1, B = 0$
 - We want $D = 1$ to be $A = 0, B = 0$; A needs to be bigger than both voltage drops (ie 1.4), so set $A < 1.4$, and B greater than 1 drop, ie $B > 0.7$
 - Is it possible to satisfy both conditions? Yes. Under this configuration, A cannot be above .7, so A and B can never both be on. Transistor 2 will cut the voltage off from A
 - Transistors turn on really fast and give lots of power
- Why is PNP good for power?
 - img6
 - * Can only transfer $H - .7V$ vs the full H

4 Lecture 4: From BJTs to FETs, MOS & Static CMOS (Thursday 9/5/2024)

-

Definition 3 (Bipolar Junction Transistor - BJT). *Due to two junctions inside*

- How to describe? With two diodes
- How to control current, turn on/off? Provide voltage to the base, which removes the depletion region. We are applying current to remove the region, which is why BJTs are called *current control devices*
- What happens when you get voltage + current? Power, aka heat. We don't like power use. This is why we don't like BJTs. Because they are current controlled devices, they need a decent amount of current power, so there is a minimum amount of power you can hit
- We don't like heat because:
 - * It will cause excitation
 - * It will induce mechanical failure
 - * Volume/heat will grow cubically, but the removal of heat is by a squared factor
- We want something that doesn't need so much current (so not a current controlled device), so instead we will use a field-controlled device, leading to *FET's*
- Reduce current to reduce power
 - 
 - If we introduce an insulator, we reduce current but no longer have a way to control the gate easily; how can we apply an electric field to remove the dead region?
 - * Capacitor
 - * Add a conductor on top of the insulator
 - * Will briefly get flow. Insulator will get excited
 - * The 5V pulls electrons in. If they are flowing up, the plate gets positively charged. The insulator then becomes negatively charged, but the insulator prevents charges from moving so it builds up and creates a tiny layer of N-type; we built a wire
 - * N all the way across so it will flow great; took very little current, hence this is a field effect (and not resulting from current)
 - * Depletion not removed, simply building a conductor where we want it
 - * Thus, power is lower due to lower current
- So how can I build the insulator layer?
 - Thin oxide layer? Silicon dioxide layer, titanium dioxide layer
 - Can oxide just in one area to build a *metal oxide semiconductor (MOS)*
- The same can be done for a PNP
 - 
 - Instead, connect to ground; positive/negative sides switched
- BJT can be faster
-

Definition 4 (Metal Oxide Semiconductor - NMOS). *NPN*

- Power is "on", ground is "off"
- Better at switching ground (similar to how NPN is)

-

Definition 5 (PMOS). *A PNP*

- Negatives on top allow it to work, hence the dot on top of the circuit symbol
- Ground is "on", power is "off"
- Better at switching power (similar to how PNP is)
- Ideally we want to build a circuit that supplies 1's and 0's (because NMOS/PMOS are better with power and ground)
 - So we want things flowing into the gate; charge or discharge the gate
 - Just like BJTs, to use the base, we need to hook up power or ground. We used pull-up resistors then eventually built a totem pole. For MOS, we are going to first build something using transistors
 - * The power network above will supply all 1's, the ground network above will supply all 0's
 - img3; static CMOS logic
 - Don't want both on at once; this will result in a short (no resistor so power will go to ground); we don't want a resistor slowing things down
 - Also don't want both off
 - * Build with a Karnaugh Map (K-map)
- img4
 - Step 1, what's the power?
 - * Power on PMOS, so power only flows when both are on, so this is an AND of some case due to the series of two
 - * Draw the K-map
 - Top-left is the power network; where is the ground network?
 - Don't want Z's, don't want a short, so where do the zeros go? Everywhere else
 - Powers of two have to be aligned horizontally (h1) and vertically (v1)
 - So what are the vertical 0's? How do we hook up NMOS to get a 0 when A is 0? Connect NMOS to A, B separately
 - * This is a NOR gate
- img5
 - Now let's build a NAND gate
 - * PMOS, since if B or A is 0, we want to supply power; connect A, B to PMOS
 - * Independent, so we can connect to power separately
 - * Need both A and B on, so they need to be connected in series to be connected to ground
- img6
 - Notice the flip/complement between A/B/C in the PMOS and CMOS being in parallel and series

5 Lecture 5: Static CMOS Timing & Sizing (Tuesday 9/10/2024)

- Static CMOS: called static because the signal stays the same / stays valid across all time
 - Slow; pullup/pulldown operate at different speeds
 - * Down is significantly faster due to NMOS being used for the pulldown network; the primary charge mover is the electrons
 - * Note: voltage is related to time, so we want lower voltage
 - * Charge carriers are slow when dealing with holes, so we need more charge carriers, but to get more, we need to make it "bigger", and making it bigger leads to higher capacitance (C)
 - * In general, to make it more reasonable, the capacitor needs to be bigger for PMOS gates; approximately 3x bigger; unfortunately, a bigger capacitor results in taking more time to turn on (ie full state), so there is not much you can do to make it faster
 - *Good for transferring power, but big and slow*
 - To accelerate, we ideally want to get rid of the pull-up network; if we do, how do we get power into the network?
 - * NMOS? It doesn't transfer power flow well; very inefficient. Generates lots of heat with minimal benefit ($\sim 1V$ loss)
 - * BJTs used totem poles to speed up (via transistors); what supplied power beforehand? A pull-up resistor, but that is also slow
 - * Capacitor? We could have a capacitor with a charge on it; the disadvantage is it needs charge time and needs to discharge, but we can make it small and thus fast
 - *[hand-drawn diagram]*(dynamic logic)
 - * Directly connecting as is, it will pull down to ground and discharge to ground, so output is $f = 0$
 - * If there is no path to ground, the output will see the charge on the capacitor
 - * This only lasts as long as the charge lasts; this results in a dynamic network and only works if it is fast
 - So how do we get a charged capacitor?
 - * Use a clock; create a charge cycle and an evaluation cycle
 - We need to supply power so we still need to add a PMOS, but we will only add one; hide its slowness by connecting it to the clock
 - * When clock = 1, PMOS is off, so no power comes in and we just have the evaluate cycle
 - * When clock = 0, PMOS is on so power flows in, so we have charge
 - * This is almost 3x faster
 - * Footless
- When charging, there is still a path to drain; ideally we can turn off the pull-down network, so we add a "foot": an NMOS to ground with a clock
- So can we cascade it for multiple levels of logic?
 - It will result in too much loss via splitting, so we can only do 1 level for most circuits like this
 - Need something that we can cascade

- Add a buffer at the output; the fastest/easiest buffer is an inverting buffer
 - Now there are no problems with fanning and no problems with discharge; this fixes the cascade problem
 - Adding the buffer turns it into **Domino Logic**
- Moving back to static (but different) logic...
- *[hand-drawn diagram]*(transmission gate)
 - For PMOS, we have a weak 0
 - For NMOS, we have a weak 1
 - Adding S, we get a flip; so the two seem more compatible now
 - What about if we connect them? We get a strong 0 and a strong 1
 - This is a **transmission gate**
 - Fairly symmetric; left/right, right/left
 - So what can we do with this transmission gate?
 - * Hook it up so inverses are automatically applied?
 - * Tri-state gate; two forms. The control gate keeps it open or closed, and the third state is Z
 - * Great for a bus situation: controller / communication
 - A mux is built with these types of transmission gates
- K-maps:
 - 1's $\rightarrow$ SoP $\rightarrow$ NAND
 - 0's $\rightarrow$ PoS $\rightarrow$ NOR
 - If we flip the 1's and 0's, we can still do the above
 - * We will still get SoP, but negated, so:
 - 0's/1's $\rightarrow$ nSoP $\rightarrow$ NAND
 - 1's/0's $\rightarrow$ nPoS $\rightarrow$ NOR
- Mux technique examples
 - *[hand-drawn diagram]*
 - We can build a bigger mux from smaller muxes; cascade
 - For the second example, the control is $\{AB\}$
 - This doesn't protect against glitches, while K-maps do

6 Lecture 6: Design Techniques — Canonical Forms, Decoders, Muxes (Thursday 9/12/2024)

- Recap / review over design techniques
- Don't simulate if you don't have an estimate, so that you can validate the simulation
 - Example: let $x = \pi$ and run the iterations

```

for i = 1:52
    x = sqrt(x)
for i = 1:52
    x = x^2

```

- Solving for x results in e , not π . This comes from a problem in precision: we know in advance the solution should be π , but the computer tells us it is e . Scientific computing / numerical analysis is the source of this example.
- Ways to design:
 - Boolean algebra
 - DeMorgan's Law
 - K-maps
 - Mux
 - Decoders
 - Decoder–encoder couple design (only efficient for one type of problem)
 - Truth table (best for ROM design)
 - * FPGAs turn everything into truth tables and store them in memory, but this only works if synchronous
 - Canonical forms
 - ROM / lookup table
- Start by making an example to design (the function $f(a,b,c) = \sum m(1,4,5,6)$):

a	b	c	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

- Consider (a,b,c) as an address; this is now a lookup table or ROM (read-only memory) design
 - Requires large, stable memory; memory look-ups are also fairly slow
 - Going from one address to the next, there can be glitches in the look-up, which is why it needs to be synchronized
- Minterm, Maxterm
 - Minterm (AND): 1 in that location, 0 elsewhere, so "minimum number of 1's"
 - Maxterm (OR): 0 in that location, 1 elsewhere, so "maximum number of 1's"
 - $f = m_1 + m_4 + m_5 + m_6 = \sum_{a,b,c}(1,4,5,6) = \prod_{a,b,c}(0,2,3,7) = M_0M_2M_3M_7$
 - * Sum of products for minterm m_i , product of sums for maxterm M_i
 - Both are circuit representations; complements of each other
 - SoP uses NAND gates, PoS uses NOR gates

- Proving only-NAND-gate usage
 - * *[hand-drawn diagram]*
 - * WLOG; the proof is equivalent for proving only-NOR-gate usage in PoS
- We have now shown circuit design with ROM/LUT, canonical form, and truth tables
- Decoder design
 - *[hand-drawn diagram]*
 - Assumes binary input; let's say 2 bits for example
 - With a decoder, we technically have minterms built into the design
 - *[hand-drawn diagram]*(example from before)
 - Decoders are just canonical form
 - Decoders are just a device
- Mux design
 - Same example
 - Break into groups of two
 - *[hand-drawn table]*
- K-maps
 - *[hand-drawn diagram]*
 - We can set up a K-map in any permutation, but there is one best way
 - Encirclements are the outputs of an AND gate
 - Increasing an encirclement can remove a glitch
 - Design for max safety; determine via usage (iedon't remove a gate / reduce an encirclement if the use case is important)
 - For product of sums, remember to apply the complement (')
- Boolean algebra: don't use, too impractical
- Decoder–encoder:
 - Build from binary to Gray code
 - *[hand-drawn table]*
 - Good for converting code; handling code switches

7 Lecture 7: Online Lecture Video (Tuesday 9/17/2024)

- Online lecture video (no in-class notes).

8 Lecture 8: Latches, the SR Latch & Race Conditions (Thursday 9/19/2024)

- Asynchronous
 - Introduces a range where an event can occur
- Synchronous

- Going to truly discrete, we want edge triggering; an example is the clock changing from 0 to 1 and 1 to 0
- *[hand-drawn diagram]*(SR latch)
 - Basic memory form
 - “Memory” = there is a feedback loop
 - 1 feedback loop, so only 1 bit of memory
 - Synchronous sequential system
 - For (0, 1): Q is 0; Q' depends on Q but eventually becomes a 1 because of the delay
 - Call it R because it “resets” the system
 - Call it S because it “sets” our system / memory
 - When both (0, 0), this is equivalent to doing nothing: take the initial condition of Q/Q' . This only works if Q and Q' are opposite of each other. This is the “stable” case.
 - q is the previous state; Q is the next state / output
 - When both (1, 1), we jam both the on and off buttons, getting 0 for both Q and Q' ; this violates the memory-hold position
 - * Nothing is wrong with it electrically, it is simply unsafe. We can just release one of the buttons to revert to a safer position
 - * Switching to (0, 0) again, we get output (0, 0) again; we created an oscillator
 - * Both not switching perfectly together but close; results in a beep frequency
 - * After some time of oscillating, it will eventually “randomly” latch
 - * First example of a race condition
 - Don’t know the output; we have two signals constantly changing
 - * Has been used for RNG, but it is not totally unknowable. How hot something is can affect the output; it is not perfectly 50/50 at every temperature range
- *[hand-drawn diagram]*(characteristic table)
 - From this table, can we determine S , R ? It counts: holds if 0, flips if 1
 - We want to build a consistent method
 - * We have a characteristic table; we want the reverse of this
 - * An “excitation table” to complete the table
 - This table is what we need, so let’s build / design it
 - * We see S is XOR, R is XNOR if the x ’s are 1’s
 - * We can also see S and R are both versions of an AND
 - * Note: XOR is an odd number of on’s; equivalence / XNOR is an even number of on’s
 - * Add a buffer so it has a similar delay to the NOT on the R output

9 Lecture 9: Latch-free Asynchronous Design & Critical Races (Tuesday 9/24/2024)

- Last time, we did the SR latch
- Do I even need a latch?
 - *[hand-drawn diagram]*

- Does this potentially have memory? Yes, because we have a feedback loop
- Can we figure out what the circuit does, how it behaves?
- Grab output at two places with input x
 - * y and Y are electrically connected, so they should be the same, but only eventually; it depends on how long it takes for electricity to get from Y down to y
 - * y coming in is the memory from prior; similarly, introduce z and Z with a long wire in between to allow for additional traversal time
 - * Will add a break
- [hand-drawn table]
 - * Remember, z will eventually become Z and y become Y
 - * Y changes with respect to the XOR gate: $Y \leftarrow \text{xor}(y, z), Z \leftarrow (x + y)'$
 - * Instead of K-maps, we are doing something different
 - * Inputs x come in very slowly compared to other things
 - * Using Gray code instead of binary, since only 1 bit flips; we can get a race condition from a state transition if using binary
 - * If a state change results in a non-stable state, we consider it a critical race
 - * In async, we need to worry about instability; we can't wait, we have to worry about races. For async, we have to remove any critical races
 - * Three possibilities: end up in the same place, end up in a different place, or loop. We only like "end up in the same place"
 - * What if I want to design a circuit without a latch?
 - We have to make a map, analyze what is stable / not stable, and remove critical races
- Methodologies
 - * Key in lock
 - [hand-drawn diagram]
 - Makes bigger stuff, but simplifies things
 - Use as many enable circuits as you need
 - Example. The user requires:
 - all x and y inputs are 1
 - $z = 1$ if in state A
 - $z = 0$ if in state B
 - We realize in this example it is very simple, but more stages = slower, yet it is safe and ensures things are implemented correctly
 - * Turning an SR into a D: simply flip the input for one of the S or R inputs
 - * Primitive states
 - [hand-drawn diagram]
 - If the output (after a slash) is in the state $\rightarrow$ Moore machine
 - If the output (after a slash) is on the arrow $\rightarrow$ Mealy machine
 - Mealy is not as stable as Moore, but Moore needs more states
 - Normally at this point, pick a node and select an input to see the transitions; we may get a race, so introduce primitive states

- Primitive states are labeled with letters
- Look at stability
- C and D stay in the same state
- Do I need all these states?
- Definition of equivalent: different states that have the same outputs
- In this example, A and D are considered equivalent and can be combined
- Remove D, rewrite D's with A's

10 Lecture 10: State Reduction, Compatibility & State Assignment (Thursday 9/26/2024)

- Recap: key and lock — write everything as enables, then combine enables to get the transition. Not fastest but easy to do. We started looking at primitives; now, how do we do state reduction in general, and how do we build designs to avoid bad properties?
- *[hand-drawn diagram]*(how to reduce states)
 - Only drawing stable states (remember, circled means stable)
 - Why do we want a state that is stable everywhere?
 - * Maybe an end state, and something that resets the circuit to run it again
 - * Maybe an error state
 - In most cases, we only worry about transitions that involve races
 - * At a , only worry about 01 and 10
 - Now we have a transition; let's convert it into a more typical graph
 - Use "—" to indicate it is just for transition (ie cell a , 01)
 - For cell a , 11, we leave it blank because we can do whatever we want with it
 - Only fill in cells adjacent to stable states that have a direct path to those stable states, assuming they have some transition involved
 - * (iee-00, on the path from f to a)
 - * (iedon't worry about d -01)
 - Blank spots are spaces we don't want to happen
 - We have not encoded spots where there are race conditions
 - From this first table of primitives, can we identify any reductions? Not yet
 - So we ask, do we need all these states?
- *[hand-drawn diagram]*(brute-force reduction)
 - Check if compatible: $\times$ if not compatible; if undecided, write extra conditions to enable compatibility (cascading)
 - * b and a are compatible; we don't have to merge, but theoretically can
 - * outputs match, states match
 - * for a compared to e , outputs are fine but states are currently unsure; $b \neq e$, but it is possible they merge, so this is a condition
 - * for a compared to f , not possible; $a \rightarrow f, 0 \rightarrow 1$, not equal
 - For the next pass, we can look at the conditions

- * *be*: is this compatible? $\times$ marked, so no
- * *cf*: is this compatible? No
- * in this example, all conditions are set to incompatible
- This is pair-wise compatibility, not complete compatibility
 - * turn this into a compatibility graph
- *[hand-drawn diagram]*(compatibility graph)
 - Draw lines between states if there is a check mark
 - DE can be merged; ABC can be merged
 - Another way to merge is: AD and BC
- Let's go back to the first table, merge and reduce
 - Let's merge and call: $ABC \rightarrow g, DE \rightarrow h, f \rightarrow f$
 - Now we can decide what to do with the non-stable states
 - Can we turn this into a Moore machine?
 - * If in H, is the output constant for the state? No; the other two can be Moore states though. H has to be a Mealy state
- Now let's do a design. In what way can I encode states? We have to turn things into numbers
- States
 - Binary: 00, 01, 10, 11
 - Gray code: 00, 01, 11, 10
 - * We get a race if we move from F to G if $G=00, F=11$. In this example we don't have to worry about this, so Gray code is a good use here as well. Simply place *g* next to *h*, and *h* next to *g* and *f*
 - One-hot: 001, 010, 100
 - * This is preferred for automated codes despite lots of unassigned / unused codes
 - * We typically add an intermediate state to slow it down and prevent a race; ieturn on the bit we are going to, THEN turn off the bit we are coming from
 - * When writing the state table, we add extra rows that are considered intermediary
 - * We can't use 000 as a transition state, because everything will be going there and we are no longer able to differentiate; it would reduce states but introduce too much ambiguity, and we are trying to remove ambiguity
 - * Use one-hot if there are few states
 - * We have to use two transitions, so if we are only allowed one transition, we have to use Gray code

11 Lecture 11: Glitch-free Design & Timing (Tuesday 10/1/2024)

- Last time, we discussed async
 - Finding the smallest number of states and identifying critical races (or other things we needed to remove), then picking a way to assign numbers to the states (ie Gray code, binary). From here it is trivial. We are now back to combinatorial logic design
- *[hand-drawn diagram]*(small design walk-through)

- K-map finds the largest encirclement
- If we did a lookup table (even a mux)
 - * A mux will introduce glitches everywhere; only the max encirclement will avoid this (so K-map, QM, Espresso)
- Want a glitch-free design in any async-related design
- For async, remember to include the safety encirclement; this allows no momentary drop / glitch between the encirclements
- Timing
 - With async, we worry about how long things take to update, so how do we measure the timing?
 - * Typically deal with a timed circuit
 - *[hand-drawn diagram]*
 - * RS dangers: race condition (1/1 state)
 - * JK: removes the 1/1 and turns it into something useful. Add logic in front to turn an RS flip-flop into an RS with 1/1 flipping memory
 - * We always have an event
 - * Difference between software / hardware programming: hardware is event-driven. The I/O is the event list
 - * $\&\&$: logic comparison; $\&$: bitwise
 - * if can be equivalent to a mux; Verilog / HDL does this. It builds the circuit from commands
 - * To be a good hardware designer, think "how can I write this so it's easy to infer what I want?"
 - Programs are equivalent to hardware
 - Timing = how long the implied circuit (from code) takes to do its work
 - * For now assume sync, so we have a clock going into memory; this is a timing reference
 - *[hand-drawn diagram]*
 - * Say we have a flip-flop. It has two gates, so it can take up to two gates worth of logic to get to the end
 - * What happens if we want to set the output to be true and we run into some change / glitch? If there is enough time, who cares. If not, things are unstable for a little. So let's add a period where set/reset cannot change during the clock cycle
 - * $T_{clk}, T_{setup}, T_{hold}$
 - Period of stability
 - * Clock speed needs to be great enough to safely store the result:
 - $T_{clk} > T_{hold} + T_{setup} + T_{elsewhere}$
 - * Let's look at T_{hold} : the simplest thing we normally do is pass things through with an inverted clock
 - $T_{clk \rightarrow queue} > T_{hold}$, so sometimes we use the max of the two
 - * Time of flight T_f
 - Light is actually slow; a good PC runs around 30 GHz, ie 0.33 ns; light traveling from the ceiling down $\rightarrow$ 30 PC cycles

- Material is important, because the time it takes to go down wires depends on the wire
- If we have one constant RC, the time constant tells us the time to reach the e value: 63% of the goal
- Typically use 10/90; the change range is 10–90 (latch below or above 10/90)
- For an FPGA, typically the memory and lookup are in the same block, so ToF is the same
- $T_{everything} = 2T_f + T_{comb} + T_{skew}$

12 Lecture 12: Clock Domains & Introduction to Verilog (Thursday 10/3/2024)

- Request a Nexys4 board
- Modern FPGAs are very large; break up the chip into regions handled by different clocks
 - Sending a signal between regions (controlled by different clocks) is difficult to get the timing right
 - * The safest thing is to set up a double buffer on both sides
 - * No timing issues
 - * Force onto the border
- Interfacing
 - EDA Playground
 - * Good code
 - Two divisions of Verilog
 - * 1991/1995 Verilog (don't write code in this; equivalent to C)
 - * 2001/2005 Verilog (equivalent to C); technically Verilog ends at 2005 and changes name
 - * SystemVerilog (equivalent to C++)
 - * Faster to write and learn in Verilog vs VHDL
 - VHDL is very good for documentation
 - Verilog is a design language; it has PMOS/NMOS, and you can build with the transistors. It works at the transistor level
 - Verilog is also parallel
 - Begin with the most basic construction
 - * Module; similar to a class
 - Instantiation convention: `.a(extb)` → pass from internal a to external b; the internal has the `.` in front
 - Shorthand: `.a ↔ .a(*)`
 - * Datatypes
 - Wire: does not hold a value, continuously driven
 - can be driven by a wire or output, but a register only by an output
 - a reg can sample, though, because we are telling it when to read
 - use the statement `assign a=d;` — this is the same as soldering, it stays forever

- Wire can be thought of as a subclass ("net" is the superclass of register and wire)
- Register: can hold a value, but on the software end. No implied memory is built; it is a way of expressing things
- Written as procedural code
- `initial` (only in the beginning); only used for test benches, sometimes used to set values
- `always` (keep doing this until X); a permanently existing thing, not just an infinite loop. This is synthesizable
- (SystemVerilog only) `always_comb`
- (SystemVerilog only) `always_latch` — only used when doing async design; if (a) b=c; (*hand-drawn diagram*: if builds a mux). Implicitly we want to hold b; we accidentally built a latch
- (SystemVerilog only) `always_ff`
- Always use the `always_` version
- `always@x`
- Typically we only use `comb/ff`; `ff` is only edge-triggered, so you need to specify `posedge` or `negedge` to trigger
- Fork/join: forcing things to happen simultaneously; not often used

13 Lecture 13: Blocking vs. Nonblocking, 4-State Logic & the Barrel Shifter (Tuesday 10/8/2024)

- Adding / shifting; we can get other basic operations
 - Looking to make something simple and quick
 - In Verilog, we will add the `generate` construct
 - * A command for how to build

```

wire a;
wire b;
assign a = b;      // this is fine
always_comb
  b = a;           // also fine: watches a, assigns when a changes. This is "blocking"

```

- Functionally there is no difference in timing between the two, due to simplicity. The difference is only in how they are written
- We can also assign this way:
 - `b <= a;`
 - The difference is that the regular way is "blocking", this is "nonblocking"
 - * Blocking = sequential; nonblocking = parallel

```

reg a, b, c;
always_comb
  a = 1;
  b = 2;
  c = 3;
  b = a;
  c = b;

```

- What happens here?
 - The value for all should be 1
 - If we instead did:
 - * `b <= a;`
 - * `c <= b;`
 - * They are done in parallel, so $a = 1, b = 1, c = 2$
 - * Samples before the clock, assigns after
- If you're in a comb block, only ever use blocking
 - Inferred assignment
 - Do a regular assignment
 - In a flip-flop / clocked block, do the other (nonblocking)
- We've talked about different states
- 4-state logic: 0, 1, x, z
 - Wires, regs, and logic are 4-state logic
 - x can represent "don't care"; combining two states to abbreviate when the bit value doesn't matter, but this is a design intent. To build, we have to pick
 - x can also represent "disconnected"; we will use z for this
 - x can also represent "don't know"
 - x can also represent "shorts"
 - Truly, it represents "don't know", which covers all the above cases
 - * if you see x's at the beginning, it means you didn't initialize anything
 - z will be "open circuit" or disconnected
- 0, 1 → bit/byte/short/int/long
 - There's also a 4-state version of int
- Genvar
 - Generate: it's a thing to build other things, so it needs its own variable
 - This is the variable to count the stuff in a generate loop
 - Not buildable or simulatable (itself)
- **Barrel Shifter**
 - Efficient bit shifter
 - Has both left-shift and right-shift
 - *[hand-drawn diagram]*(3 subtypes of shift)
 - * These tell us the convention for the value being shifted in
 - * Logical: 0 shifted in
 - * Arithmetic: preserves the sign bit for a right shift; left shift still shifts 0's in
 - Left shifts are the same as logical, so there won't be a distinction in certain coding languages
 - * Circular: MSB/LSB wraps around; preserves the original number, often used in signal processing

- * Use 2's complement: identical inside the adder for negative numbers
 - This is why for logical, we extend left by the MSB, right by 0
- We can make a universal multiplier, but we can't do division due to negatives
- Shifting truncates, not rounds
- We can do multiple sequential shifts to shift by more than 1
 - * $O(n)$ complexity in time; slow, but $O(1)$ space
 - * We can in theory build every possible configuration to make it $O(1)$ in time, running each configuration into a mux, but $O(n)$ space
 - * We will find a solution in between: $O(\log n)$ space and time
 - Use a binary tree (in general, use one if you want $\log n$ for anything)
- Barrel shifter: $O(\log n)$ time, $O(\log n)$ space
 - * We will generate it via a binary tree; think of a cipher barrel visually
 - * Barrels with shifts of 1, shifts of 2, shifts of 4, ..., shifts of 2^n
 - * This is a mux, built into a tree
 - * For each barrel, indicate if we want a shift of that amount
 - * *[hand-drawn diagram]*
 - * For the generate statement, we can just tell the machine the pattern (see the module below). How might you systematize this?

```

module cBS #(parameter N=8, LOG2N=$clog2(N))
  (input logic [N-1:0] in,
   input logic [LOG2N-1:0] shamt,
   output logic [N-1:0] out);
  logic [N-1:0] a [0:LOG2N];
  assign out = a[0];
  assign a[LOG2N] = in;
  genvar i;
  generate
    for (i=0; i<LOG2N; i++) begin: tumbler
      assign a[i] = shamt[i] ? {a[i+1][2**i-1:0], a[i][N-1:2**i]} : a[i+1];
    end
  endgenerate
endmodule

```

14 Lecture 14: Nexys Board Setup & Building the Barrel Shifter (Thursday 10/10/2024)

- Nexys board instructions
 - Google: nexys4ddr xdc
 - Download the entire file
 - Launch Vivado 2024.1
 - Create a project
 - * RTL (register-transfer level)
 - * Don't add sources
 - * On "default part", look up the board name under the "board" search

· OR xc7a100Tcsg324-1 under parts (on the back of the box)

- File → Add Source
 - * Add file
 - * Select the downloaded file
- Turn on/off the switch
- Segment name: ABCDEFG
- If the anode and segment are both engaged, it turns on a particular segment
 - We could in theory wire each individually, but that would take 64 signals / 64 pins
 - In this way, only 16 are needed
- Currently, uncommented LEDs and switches
- Uncomment the clock, because you cannot run an FPGA without a clock
- Do not mix 3.3 V / 5 V standards in the same block
- Generate bitstream → open hardware manager
- **Now: programming the barrel shifter**
 - We will talk about how to build it, then build it with the generator
 - How do I build a shift (divide by 2)? How do I build this?
 - For example, do a circular shift
 - Only need wiring, so how do I reorder wires? Change the sequence, renumbering; re-ordering *a* to become *b*
 - Use {} for rewiring. Assume wire[5:0]:
 - * {a[0], a[5], a[4], a[3], a[2], a[1]} or {a[0], a[5:1]}
 - * Comma-separated list; {} means to append all together
 - * We can now stick this (*b*) into a mux, with the other end of the mux being just *a*
 - How to specify the mux?

```
if (shamt[0]) c = b; else c = a;           // if-statement form

c = cond ? true : false;                  // ternary form, one line
c = shamt[0] ? {a[0], a[5], a[4], a[3], a[2], a[1]} : a; // or {a[0], a[5:1]}

// In general, for a shift of N, the index is N-1:0, max-1:N
for (i=0; i<log2(max); i++)
    c = shamt[i] ? {a[2**i-1:0], a[max-1:2**i]} : a;      // first row is 0:0
// N = 2^i = 2**i
```

- We still need to figure out the naming convention for the output, since the previous output goes into the next input

15 Lecture 15: Parameterized Barrel Shifter with generate (Tuesday 10/15/2024)

- Last time, we talked about parallel shifters
- We built it with the ternary operator, built a table, indexed the table, then rewrote the operation only in terms of i and a
- Moving on: how do we make a change without only using a 's and c 's?
 - We can't use a temporary variable; that would mean many things dumping into a single wire
 - We have to exhaustively enumerate every time it's reused, so the next question is "how many times is it going to be reused?"
- *[hand-drawn diagram]*
 - in is a , out is c ; we will replace them once we figure it out
 - We know sh is the shift amount, shift[i]
 - The beginning and end will each need a name
 - In between, the names "in" need to be the same as the names "out", so this implies in/out all need to share one name
 - We can't just make them all a 's; that means everything is also hooked to itself
 - * We need to make an array; we can call it a , but we need to index it
 - * Space complexity: counting the number of wires (or the number of boxes +1)
 - With 4 boxes, we have up to 15 shifts, so we can handle 16 bits (0 to 15)
 - * With 3, 8 total bits are handled
 - * $\log_2(N) \rightarrow$ how many boxes are needed
 - We will use logic: simply better than wires and registers
 - * We're trying to build something but don't have the local parameter available, so we can't use it to generate. It needs to be passed to the generate

```
logic [bits-1:0] a [$clog2(bits):0]; // "[...]" after the name is an unpacked array
                                     // (a C-math library typically has a $c in front)

// Example instantiation -- need to pass both bits and log2(bits):
module BS #(parameter N=8, LOG2N=$clog2(N))
  (input logic [N-1:0] in,
   input logic [LOG2N-1:0] shamt,
   output logic [N-1:0] out);
// BS #(.N(16)) bigBS(.in(), .shamt(), .out()); // instance name comes after
```

- Example of the generate module (below). Inside the generate, it will know how to make everything and stack it all:

```
module cBS #(parameter N=8, LOG2N=$clog2(N))
  (input logic [N-1:0] in,
   input logic [LOG2N-1:0] shamt,
   output logic [N-1:0] out);
  logic [N-1:0] a [0:LOG2N]; // gives log2(n) wires
  // hook a[0] to out, a[LOG2N] to in
  assign out = a[0];
```

```

assign a[LOG2N] = in;
// names made; just need to generate
genvar i;
generate
  for (i=0; i<LOG2N; i++) begin: tumbler // naming the assign; name appears in simulation
    assign a[i] = shamt[i] ? {a[i+1][2**i-1:0], a[i][N-1:2**i]} : a[i+1];
  end
endgenerate
endmodule

```

- So can you make a barrel shifter on your own (bs-1s)?

16 Lecture 16: Board Interface & Simulation Setup (Thursday 10/17/2024)

- For the report, just describe the methodology: what you are doing and the process; call it "Method". What are we building, how did you approach it?
- Results: show how we tested it
- Also want the original files for the code

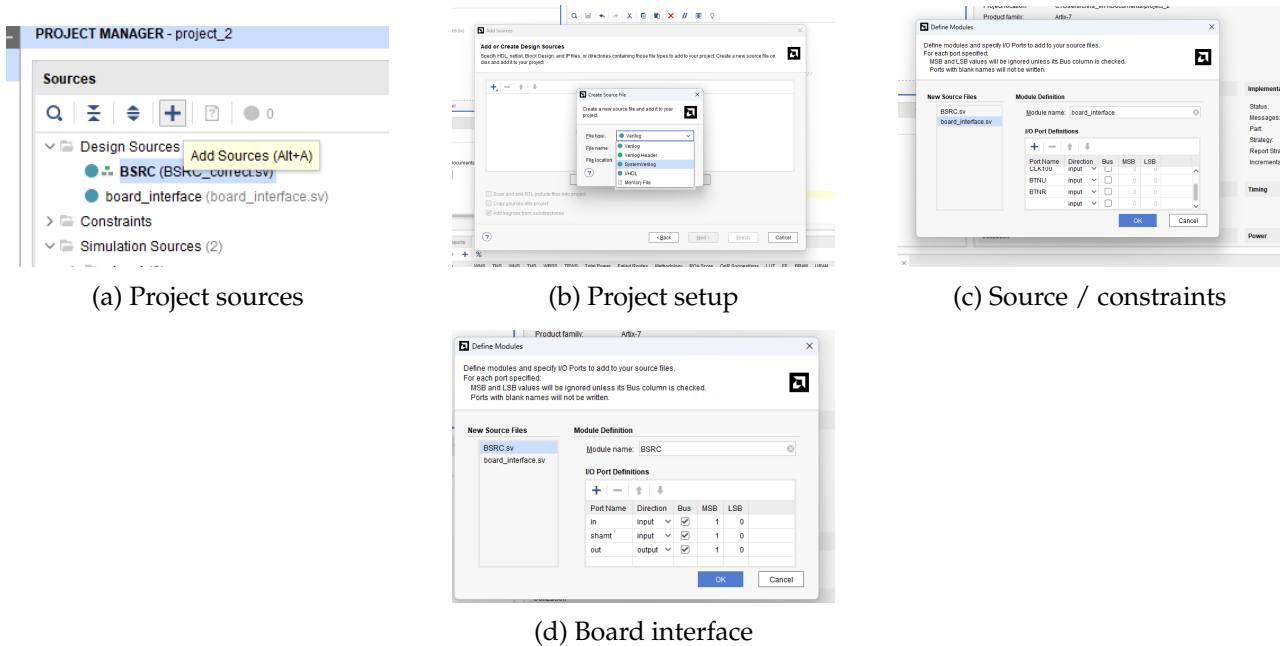


Figure 6: Vivado project setup for the barrel shifter.

- Now: we need to specify building a flip-flop in the board_interface file

```

always_ff @(CLK100MHZ) begin // CLK100MHZ is the trigger
  // specifying time, so we want a nonblocking assignment
  // will need 2 different variables
end

```

- Now, doing simulation and setting up the test board interface file
- Next, we need to do the sequence of events you want it to perform to fully simulate

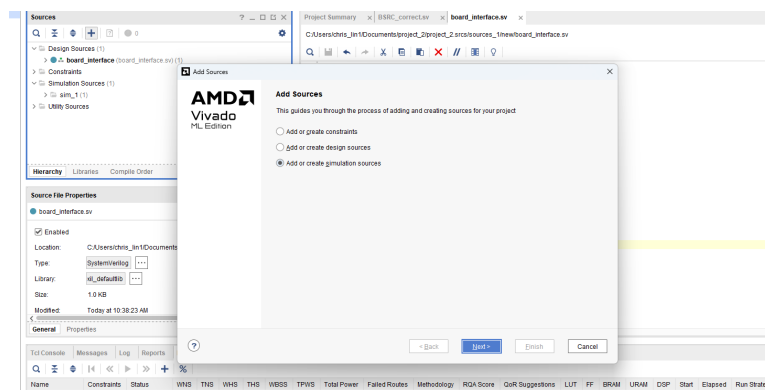


Figure 7: Flip-flop in the board interface file.

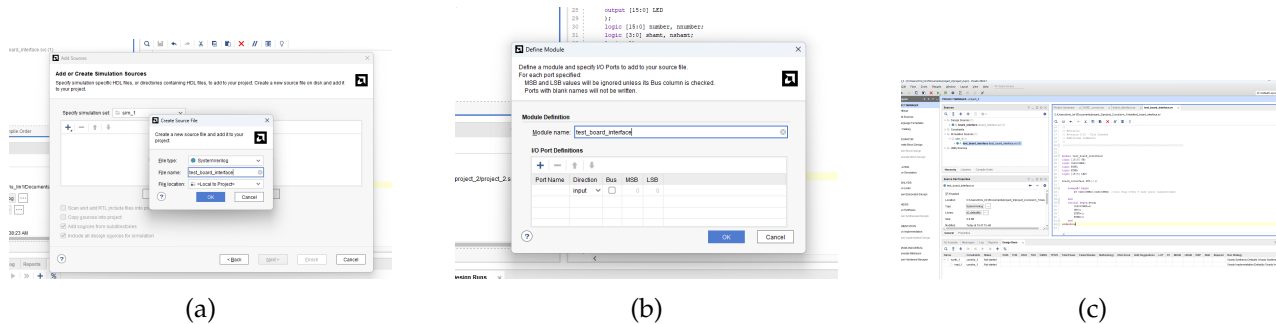


Figure 8: Setting up the test board interface and simulation.

17 Lecture 17: The Ripple Carry Adder (Tuesday 10/22/2024)

- Last time, the barrel shifter; we need to implement both a left and right shifter for the code
- Today, we deal with addition: the complexity of addition and how to construct it
- Ripple carry by generate statement
- Then we will follow up by building two more with different complexity
- Ripple carry: we will "ripple" the carry
- How to draw a circuit for this
 - At each stage, take the three inputs: carry, no. 1, no. 2
 - * output is a sum and a carry
 - * usually done with half adders
 - * get a partial sum, then add the extra digit to the partial sum
- *[hand-drawn diagram]*(full adder built from half adders)
 - Looking at the table, we have an AND and an XOR, so we just need one each for the half adders
 - Going from half adder to full adder shows bottom-up design; we will not do this in this class. It is much more efficient to directly build a system that does a full adder
 - *[hand-drawn diagram]*
 - * Recognize: carry out is just "2 or more 1's"
 - * $x \cdot y + x \cdot c_{in} + y \cdot c_{in}$

- * Recognize: the sum column is counting odd 1's $\rightarrow$ XOR
- First carry-in: the mode bit. Indicates if we are adding or subtracting
- [hand-drawn diagram]
 - * $c_{out} = x \& y_t + x \& c_i + y_t \& c_i$
 - * Name it y_t as "temporary y "
 - * $y_t = m \wedge y$
 - * $s = x \wedge y_t \wedge c$
 - * These 3 equations are the definition of my full adder
 - * What's the challenge of converting to generate? The names will change
 - * Mode is easy, just make it my c_{in}
 - * Sum is next easiest
 - * x, y are also easy, just with []

```

module ripplecarry #(parameter N=8)
  (input  logic [N-1:0] x,
   input  logic [N-1:0] y,
   input  logic      mode,
   output logic [N-1:0] s,
   output logic      cout);
  logic [N:0] c;
  assign c[0] = mode;      // LSB of the carry array is the mode; assign c0 to m
  assign cout = c[N];
  logic [N-1:0] ytmp;
  genvar i;
  generate
    for (i=0; i<N; i++) begin: ripadd
      assign ytmp[i] = mode ^ y[i];
      assign s[i]    = x[i] ^ ytmp[i] ^ c[i];
      assign c[i+1]  = x[i]&ytmp[i] + x[i]&c[i] + ytmp[i]&c[i];
    end
  endgenerate
endmodule

```

- Now, make the interface and test bench

18 Lecture 18: Adder Interface & Test Bench (Thursday 10/24/2024)

- Last time, the ripple carry with a generate case
 - Think about interface design and the testing module
- So how does the user send us data when building this board? What kind of inputs?
 - Switches for input (15 bits), LED output (16 bits); a second set of LEDs (the RGB ones) can be used for the carry-out
 - Upside: you see the full output
 - Downside: everything in powers of 2, because everything is a binary tree
 - Now we need to use buttons to switch mode: add or subtract
 - * Also used for each term
 - * Think about positioning for each $+/-$, A/B; we have to think about the user to make it intuitive

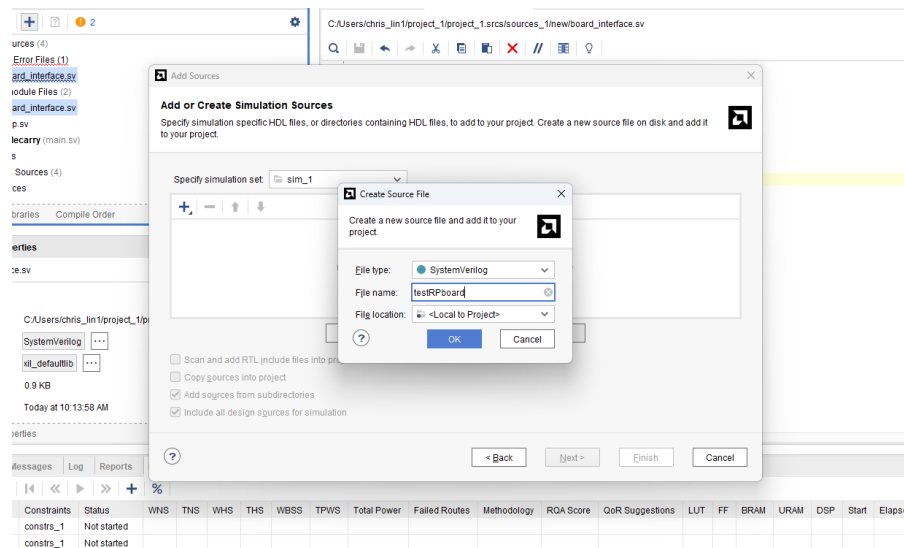


Figure 9: Ripple adder board test bench / simulation.

- Creating the simulation

```

module testRPboard;
    logic CLK100MHZ;
    logic [15:0] SW;
    logic BTNL, BTNR, BTNU, BTND;
    logic [15:0] LED;
    board_interface UUT(.*);

    // tasks: similar to functions, but also allow advancing the clock;
    // generally used in simulations to condense standard operations
    task setA(input integer A); // local variables can be declared here, outside begin/end
        begin
            SW = A;
            BTNL = 1;
            @(negedge CLK100MHZ); // wait for the negative clock edge (or could use #10;)
            BTNL = 0;
        end
    endtask
    task setB(input integer B);
        begin
            SW = B;
            BTNR = 1;
            @(negedge CLK100MHZ);
            BTNR = 0;
        end
    endtask
    task addit();
        begin
            BTNU = 1;
            @(negedge CLK100MHZ);
            BTNU = 0;
            #20;
        end
    endtask
    task subit();
        begin
            BTND = 1;
            @(negedge CLK100MHZ);
            BTND = 0;
            #20;
        end
    end

```

```

endtask

always @* #5 CLK100MHZ = ~CLK100MHZ;    // won't work yet -- need initialization

initial begin: setup
    CLK100MHZ=1; SW=0; BTNL=0; BTNR=0; BTNU=0; BTND=0;
    // careful: these numbers are read in as decimal
end
initial begin: test
    // test example A=15, B=7 for both add and subtract
    setA(15); setB(7); subit; addit;
end
endmodule

```

- Now thinking about test cases:
 - Edge cases
 - Input wires
 - Output wires
 - Ops or no-ops
- Randoms (will talk next time about getting this done)

19 Lecture 19: The Carry Lookahead Adder (Tuesday 10/29/2024)

- Last time: ripple carry adders, how to test, introduced the idea of a test bench and tasks to simplify the process
- The ripple adder works, so why not use it? It's slow
 - Sequential hardware (think: a wire connecting the adders)
 - Time complexity: $O(N)$, i.e. $N \times$ the time of each full adder
 - * What is timing? Get it down to something we can measure logically; we get it down to the gates (iere-evaluate as gates)
 - * Let's put as little hardware between c_{in} and c_{out} as possible
 - So we put X, Y as inputs into the half adder to minimize the above
 - This output is a sum and an internal carry; we then take the intermediate sum and feed it into another half adder with the carry bit c_i to get the sum output
 - We look at c_{in} because it is what is fed in from the previous full adder
 - Thus, our timing is $2N$, or $O(N)$
 - Can you get $O(1)$ in theory?
 - * For any circuit, you can always write the circuit in canonical form (two levels of logic), so everything should be doable in two levels of logic
 - * This only refers to time complexity, and we get problems with wires
 - So what if we break the wires between the full adders? Then we only have two levels of half adders, with the carry-in coming in from a constant level of logic
 - * In practice, 8 to 24 is the most wires you can feed in as inputs
 - Let's consider the first bit: s_0 results from the sum of both LSBs
 - * $c_1s_0 = x_0 + y_0 + c_0$

- * $c_2s_1 = x_1 + y_1 + c_1$
- * ...
- * $c_{i+1}s_i = x_i + y_i + c_i$
- * $\rightarrow$
- * $s_0 = x_0 \wedge y_0 \wedge c_0$
 - Results in 3 inputs to one gate
- * $c_1 = x_0y_0 + x_0c_0 + y_0c_0$
 - Technically we want "2 or more" 1 bits, but this covers both
 - So we have 3 AND gates feeding into one OR gate
- * $c_2 = x_1y_1 + x_1c_1 + y_1c_1$
 - We don't know c_1 , so we take the previous expression for c_1 and plug it in:
 - $c_2 = x_1y_1 + x_1x_0y_0 + x_1x_0c_0 + x_1y_0c_0 + x_0y_1y_0 + x_0y_1c_0 + y_1y_0c_0$
 - This is two-level logic; 7-input OR gates now
- * $x_iy_i = G_i$, "generate". If both are on, then we know there will be a carry
 - So with this, we can know that in parallel
- * The carry can also propagate from a lower level
 - $p_i = x_i + y_i$, "propagate"
- * The last thing we need is the carry from the previous level
- * We then say: if propagate AND carry, then (OR-ed with generate) we will have a carry
 - $c_{i+1} = G_i + P_i c_i$
 - Either a carry is coming in and can propagate, or a new one is starting here
 - What happens if we use this recursion to expand?
 - $c_{i+1} = G_i + P_i G_{i-1} + P_i P_{i-1} c_{i-1}$
 - Gives us linear space: $2N + 1$
 - All we needed was a half adder before, to build the generate and propagate components
 - This is called **carry lookahead**
 - This is two levels of gates, then feeds into the original half adder, for a total of four levels
 - Next time, going into the block part: how to build the blocks and combine them

20 Lecture 20: Block Carry Lookahead (Thursday 10/31/2024)

- Last time, the carry-lookahead situation
 - We could build a 2-level logic, but that introduces too much (spatial / current) complexity
 - * Fan-in / fan-out: device limits on input/output
- We used generate and propagate:
 - $c_i, x_i, y_i \rightarrow c_{i+1}, s_i$
 - Without waiting for c_i , we can use x_i, y_i to determine c_{i+1}

- We said propagate can be OR or XOR, because GENERATE already takes care of one of the 1/1 scenarios
- With the half adder, we already have an XOR in it, so we want to just use the XOR
- If we didn't have a half adder, we'd use OR because it's faster. Smaller and faster, so in general we prefer OR
- We will repeat this half adder for every single gate

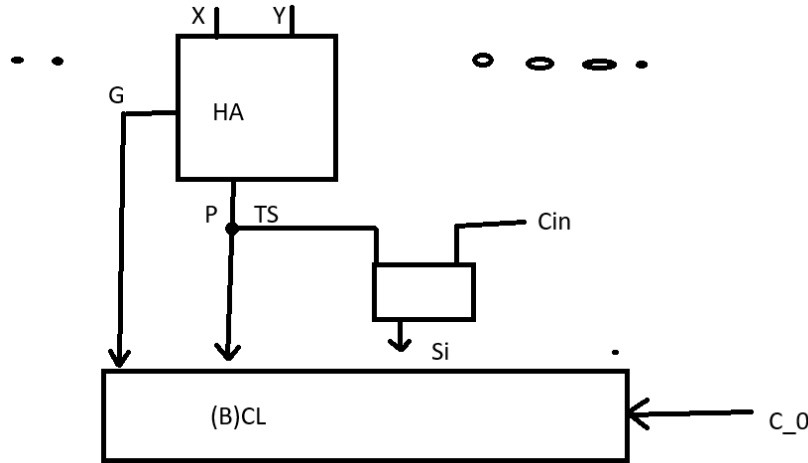


Figure 10: Full adder feeding a (Block) Carry Lookahead unit: the half adder produces G (generate) and P (propagate); the temporary sum and c_{in} produce s_i .

- (Basic structure shown above)
- Now we need to discuss (B)CL; let's look at the algebra
 - $G_i, P_i, i \in \{0, \dots, N-1\}$: the set of given values, output from the top half adder
 - We also have carry-in and mode: $c_{in}, mode$
 - $c_1 = G_0 + P_0 c_{in}$ (1)
 - * Two levels of logic here: $+$ and $\cdot$
 - $c_2 = G_1 + P_1 c_1 = G_1 + P_1 G_0 + P_1 P_0 c_{in}$ (2)
 - * Distribute to reduce the levels of logic
 - $c_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_{in}$ (3)
 - * Once again, distribute to reduce the levels of logic
 - $c_{i+1} = G_i + \sum_{j=0}^i \left(\prod_{k=i-j+1}^i P_k \right) G_{i-j-1} + \left(\prod_{j=0}^i P_j \right) c_{in}$
 - * Now we have a pattern, so we have a generate function
 - * This grows; simulation will work fine, but building it will cause problems for a large number of bits
 - * So what can we do? We need to regulate the number of wires going in; maybe a mux? In blocks?
- Doubling the bits results in 8 blocks; with this we get a constant speedup
- Let's do better: we need to predict the carries in each block. Use block carry lookahead, take the generate/propagate from each block, and use it to generate the carry for each level

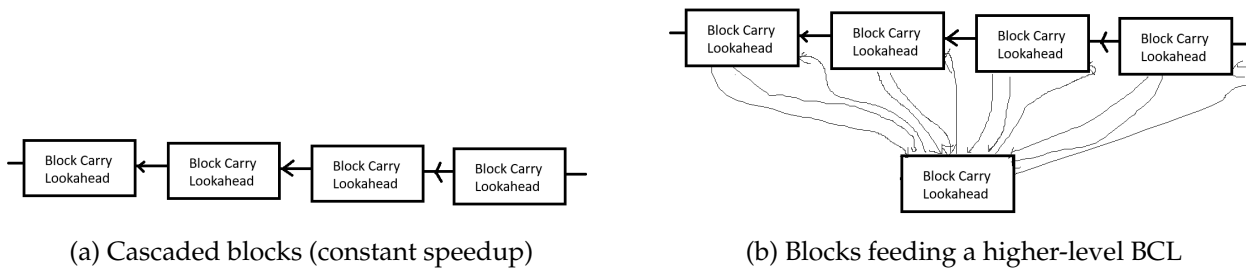


Figure 11: Building block carry lookahead: doubling bits into 8 blocks, then predicting the carry into each block.

- Is it meaningful to talk about generate/propagate at this level?
 - $c_{i+1}^* = G^* + P_i^* c_i^*$, with c^* between each BCL
 - $G^* = G_j + P_j G_{j-1} + \dots + P_j P_{j-1} \dots P_1 G_0$
 - $P^* = P_j P_{j-1} \dots P_0$
 - * So can I make a generate/propagate for each BCL?
 - * This is a tree; we finally have logarithmic complexity, and limited spatial complexity since each unit is constant, dependent on j
 - * So we can build a module and instantiate a lot of these blocks of size j

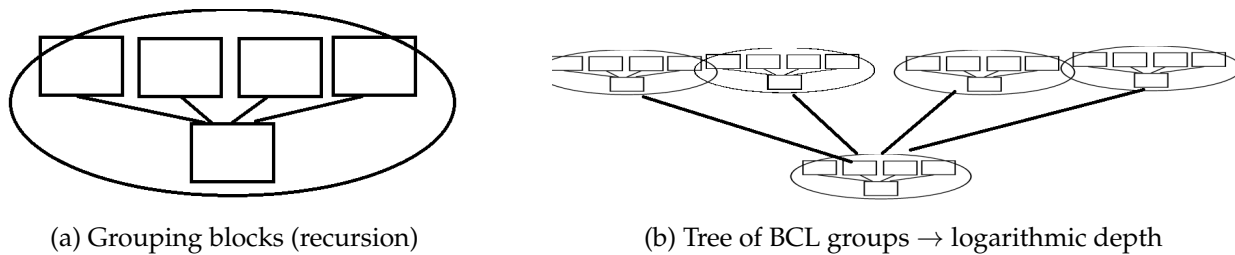


Figure 12: Treating a group of BCL units as a single BCL gives a tree with logarithmic time and constant space per unit.

- The number of bits that can be handled increases by 2^N over N layers
- So now we get logarithmic time with constant space
 - Defining complexity as the number of wires going in
- To do: build 1 block carry lookahead for 4 bits. Build and test.

21 Lecture 21: Project Design Methodology (Tuesday 11/5/2024)

- For the project, looking into the scale of the lab
 - Example: using a gyro/accelerometer, electronic levelers
- Basic idea for building
 - First thoughts (prototypes) and drawings
 - Draw a block diagram of a candidate solution
 - Work a solution, start easy

- Draw more details for the block diagram, based on the solution from step 3
- Do it
- Module for the board, submodule for the overall adder, sub-submodule for the BCLA

22 Lecture 22: Bit Reversal & the LFSR (Tuesday 11/12/2024)

- Different ways to flip
 - Using a for loop, generate + assign
 - Using << or >>
 - * Indicates which direction to read from (so >> does nothing here)
 - * Including a number after also indicates how many to read at a time before flipping (ieflipping byte order instead of bit order); <<8 for flipping byte order
 - * {} means to order

```
module test_flipit();
  logic [15:0] in, out, out2;
  flipit uut(.*);
  initial begin
    in = 1;
    // random number generation: easiest with a builtin function
    /* #20 in = $urandom; */
    #20 $finish; // trivial test; $finish ends the test
  end
endmodule
```

- Randomly generating a number: note that when randomly generating in simulation, the default seed is 0
- Next time, we will be making an LFSR (linear feedback shift register)
 - Randomization done on bits
 - Takes a linear combination of the bits and feeds it back to the beginning of the sequence
 - Use XOR because it is evenly balanced
 - Pseudo-random number generation

23 Lecture 23: The Conditional Sum Adder (Thursday 11/14/2024)

- Recently, we looked at different ways to do addition, trading off time and space, and identified three general methods: ripple carry (constant space, N time), full carry lookahead (N space, constant time), the block version which costs some more time (tree form $\rightarrow \log N$), with a middle ground in conditional sum
 - Get the functional block lookahead working to know how to stack
- Conditional sum
 - Do all calculations in advance, then merge after
 - The trick is in how you merge them (swiftly)
- Example: $0110 + 1101$
 - Method one: $1|0011$

- * The carry calculations are the problem: 1 1 0 0 _
- Instead of cascading, just do the addition, but repeat it assuming there is and isn't a carry (listed right to left, bitwise)
 - * $0 + 1 \rightarrow (\text{carry is } 1) 1|0 \quad \text{or} \quad (\text{carry is } 0) 0|1$ (1)
 - * $1 + 0 \rightarrow (\text{carry is } 1) 1|0 \quad \text{or} \quad (\text{carry is } 0) 0|1$ (2)
 - * $1 + 1 \rightarrow (\text{carry is } 1) 1|1 \quad \text{or} \quad (\text{carry is } 0) 1|0$ (3)
 - * $0 + 1 \rightarrow (\text{carry is } 1) 1|0 \quad \text{or} \quad (\text{carry is } 0) 0|1$ (4)
- The first value, the carry, will be a mux decider to determine which of the next outputs to select
- From (1), we know the output is 0|1
- However, if we just cascade, we still get linear behavior and lots of work with no gain
- Instead, perform on half every time; reduces to log in time; uses 2-input muxes and just grows in wire count ($\log n$)
 - * Combine (1) and (2) into a new pair: (1|00, 0|11) (1*)
 - * Combine (3) and (4) into a new pair: (1|01, 1|11) (2*)
- Note: at each layer, we feed more bits/wires into the mux (first 1 bit, then 2 bits, etc.)
 - * Combine (1*) and (2*): (10100, 10011)
 - * Sum = 10011
- The biggest cost is at setup time
 - In cases where the problem is small, just use the algorithm with the least setup time
- For each block:
 - Feed inputs x_i, y_i into a half adder (not a FA, because we only need a HA)
 - Group the carry and sum into a bundle: c_0s_0
 - For the second output, feed into another half adder, adding 1: c_1s_1
 - This is just 1 full adder, and we've changed the carry into a fixed 1
 - * Use the carry from both outputs, then OR (note: not possible for both to be set) to become c_1
 - * Can be OR
 - * or XOR, but OR here just because it's faster
- This is the setup; everything after is fed into a mux tree, and the very end is a selector (technically just another mux), c_{in}
 - Bits will increase by $2(i - 1) + 1$, or $2i - 1$, after each layer
 - "Selecting the bits from the left, depending on the carry from the right"
 - Note: you will get a lot of 3-input AND gates, but the number of inputs is always the same. It simply takes up more space on the board and generates more heat, but doesn't limit you, because the muxes are fixed

24 Lecture 24: File-driven Test Benches (Thursday 11/21/2024)

- Create a task that performs test cases (reading test vectors from a CSV file):

```

task addit(input logic C);
begin
    // ...
    assert (LED == ans) $display("correct"); else $warning("error");
    // ...
end
endtask
task testadd(input logic [15:0] A, B, C);
begin
    setA(A); setB(B); addit(C);
end
endtask
task readvectorfromfile(input string filename);
// use a csv file, because most programs can output csv files
int fid, code;
logic [15:0] A, B, C;
begin
    fid = $fopen(filename, "r");
    if (fid) begin
        while (!$feof(fid)) begin
            code = $fscanf(fid, "%d, %d, %d\n", A, B, C);
            if (code == 3) testadd(A, B, C);
        end
    end
    $fclose(fid);
end
endtask
initial begin: test
    #10;
    readvectorfromfile("mytest0.csv");
    #30 $finish;
end

```

25 Lecture 25: Conditional Sum Wiring & Recursion (Tuesday 12/3/2024)

- Last time, we asked how to hook up all the wires

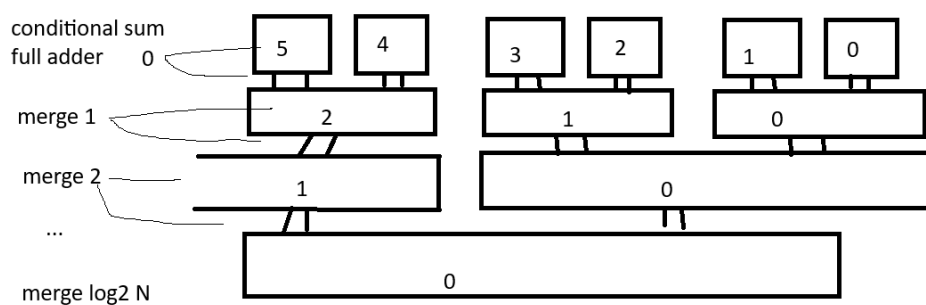


Figure 13: Conditional sum adder: half adders / conditional-sum full adders on top, then merge stages 1 through $\log_2 N$ (a mux tree).

- Reminder: each wire for the different levels has a different size; the size is $2N - 1$
- Counting devices for the top layer, we need N devices
- For the next layer, we need $N/2$ merge devices

- Keep dividing by 2 to get the number of devices needed

```
for level = 1 : log2(N)
  for devices = 1 : (.../2)
    ....
```

- You can make an inefficient build, but Vivado will delete unused wires; it is truly only wasteful in simulation
- If using recursion, you need another module. It will build only this:

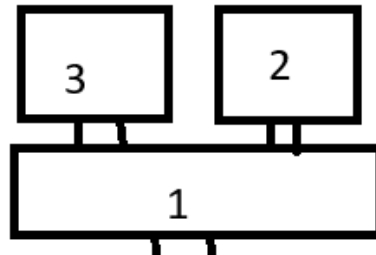


Figure 14: One merge step combining two sub-results.

```
module build
  generate
    if (lvl > 0)
      Merge ...
      R buildme(level-1)
      L buildme(level-1)
    else
      Build CFA
    end
  endgenerate
endmodule
```

- So what happens if you need to make this tight and efficient?
 - If the system can't automatically clean things up...

26 Lecture 26: The Babbage Difference Engine (Thursday 12/5/2024)

- Some numeric tricks
 - Core ideas, math, theory
 - We will build one next week (the last day of class)
- Either 2 or 4 questions on the final; 1 over initial theory, 1 over Verilog / SystemVerilog
- (Charles) Babbage
 - Worked with Ada Lovelace; did the hardware aspect of her work
 - Two major designs; we will discuss the "difference engine"
 - * Based on difference equations (discrete differential equations)
- Problem setup: $d = \frac{1}{2}at^2$; let $a = 2$ and $t \in \{0, 1, \dots\}$, so $d = t^2$, $d \in \{0, 1, 4, 9, \dots\}$
- Look at the differences:

- $0, 1, 4, 9, 16, 25 \rightarrow 1, 3, 5, 7, 9 \rightarrow 2, 2, 2, 2$
- Note that taking $\frac{d^2d}{dt^2}$ is also constant
- Remember that $\frac{d}{dt}f(t) = \lim_{h \rightarrow 0} \frac{f(t+h) - f(t)}{h}$
- So we use these difference equations; good for polynomials
- Newton did all this
- Babbage said: what if you know the initial conditions? How do you figure out the rest of the numbers?
 - Work backwards; we can propagate the 2's; reconstruct 1, 3, 5, 7, 9 via an add unit; repeat to reconstruct the first row 0, 1, 4, 9, 16, 25
 - So based on just the basic initial conditions, we can get everything
 - Call the initials d_0, v_0, a_0 ; solve simple differential equations that follow the power rule
 - We have already built addition machines, so use any of the machines for the overall machine below

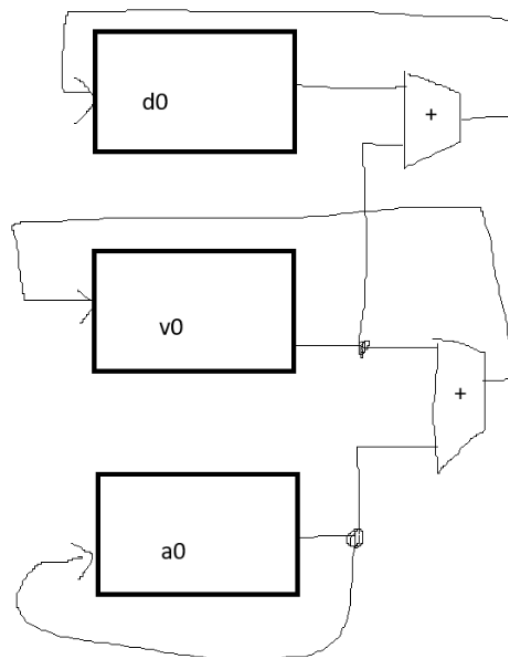


Figure 15: Difference engine datapath: registers d_0, v_0, a_0 with feedback through adders.

- We can take measurements and update it
- Now let's complicate it a little; how about an arbitrary quadratic equation that we're trying to solve?
 - $d = \alpha t^2 + \beta t + \gamma$; we want to calculate it but also perform Babbage's method
 - * Let's use $(\alpha, \beta, \gamma) = (1.625 \approx 1\frac{5}{8}, 5, 13)$
 - * To get this, we need 3 values of d ; we will do 4 to verify we get a constant
 - $d_0 = 13, d_1 = 19.625, d_2 = 29.5, d_3 = 42.625$
 - * Differences are: $(6.625, 9.875, 13.125)$, then $(3.25, 3.25)$
 - * We see we get a constant again

- Next step: how do we build the difference machine?
 - * We need something to both hold and feed back
 - Flip-flop; a multibit register
 - * Inputs d_0, v_0, a_0 are N -bit registers
 - * The adders are just one of the add options; we will start with just using “+”

```

logic [N-1:0] a, na, v, nv, d, nd;
always_ff @(posedge clk) begin
    a <= na;
    v <= nv;
    d <= nd;
end
always_comb begin
    if (restart) begin
        na = ainit;
        nv = vinit;
        nd = dinit;
    end
    else begin
        // Note: there may be some bleed-over if the timing can't be met;
        // it will latch if it can't guarantee completion in time
        // (SystemVerilog should use a latch). If timing doesn't work out,
        // it becomes a multi-cycle update. To handle adders that don't fit
        // or an uncontrollable clock, implement a counter or a control wrapper.
        na = a;
        nv = v + a;
        nd = d + v + a;
    end
end
// Don't need multiple modules; shouldn't need a generate statement either

```

27 Lecture 27: Difference Engine Implementation & Testing (Tuesday 12/10/2024)

- Header file:

```

`ifndef _BDE_vh
`define _BDE_vh
    `define BITS 16
    `define POLYSIZE 2
    typedef logic ['BITS-1:0] BDEvals_t ['POLYSIZE:0];
`endif

```

- ‘ tells us the pre-processor runs it; it happens before the “true compile”
- The preprocessor is a substitution step
- Only the preprocessor allows you to use _ at the beginning of a variable
- “_t” typically means you’re dealing with a type
- "header.sv" vs <header.sv>: a difference of location. " " is local
- New implementation with variables

- The only difficulty was that we had to make our own variable types to set this whole thing up

```

`timescale 1ns / 1ps
`include "BDEheader.sv"
module DiffEngine #(parameter N=8, DEPTH=2)
  (input  BDEvals_t      initvals,
   input  logic          restart,
   input  logic          clk,
   output logic [N-1:0] result);
  int i, j;
  logic [N-1:0] state [DEPTH:0];
  logic [N-1:0] nstate [DEPTH:0];
  assign result = state[0];
  always_ff @(posedge clk) begin
    for (i=0; i<=DEPTH; i++)
      state[i] <= nstate[i];
  end
  always_comb begin
    if (restart) begin
      // remember: this last one is constant
      nstate[DEPTH] = initvals[DEPTH];
      for (j=0; j<DEPTH; j++)
        nstate[j] = initvals[j];
    end
  end
endmodule

```

- Now for the testing (we use BDEclear to technically do a state reset):

```

task BDEsolve(input logic [N-1:0] D, V, A, input int T);
  begin
    @(negedge clk);
    dosyscheck = 0;
    initvals[0]=D; initvals[1]=V; initvals[2]=A;
    restart = 1;
    @(posedge clk);
    restart = 0;
    for (i=0; i<T; i++) begin
      result_TRUE = 0; // this is the precalculated reference
      @(negedge clk);
      @(posedge clk);
    end
  end
endtask
task BDEclear(input int T);
  begin
    BDEsolve(0, 0, 0, T);
  end
endtask
task BDEverify(input string filename);
  begin
    int fid, code, T;
    fid = $fopen(filename, "r"); // the $ interpreter runs this
    if (fid) begin
      @(negedge clk);
      dosyscheck = 0;
      code = $fscanf(fid, "%d,%d,%d\n", initvals[0], initvals[1], initvals[2]);
      restart = 1;
      @(posedge clk);
      restart = 0;
    end
  end
endtask

```



```

        code = $fscanf(fid, "%d\n", T);
        for (i=0; i<T; i++) begin
            code = $fscanf(fid, "%d\n", result_TRUE);
            @(negedge clk);
            dosyscheck = 1;
            @(posedge clk);
        end
        code = $fscanf(fid, "%d\n", result_TRUE);
        $fclose(fid);
    end
end
endtask
initial begin
    sysgood = 1;
    correct = 1'bz;
    forever begin
        @(negedge clk);
        if (result == result_TRUE)
            correct = 1;
        else begin
            correct = 0;
            if (dosyscheck) sysgood = 0;
        end
    end
end
end

```

- A way to watermark your code is a no-op: sticking in an operation that isn't stored